Cairo University
Faculty of Engineering
Electronics & Communications Dept.

Neural Networks — Homework 1

Derivation of the Perceptron Learning Rule

Prepared by:

Abdelrahman Hatem Nasr ID: 9220461

Section: 2

Submitted to:

Prof. Dr. Mohsen Rashwan

Dr. Mohamed Abdelghani

April 2026

Step 1. We have an input vector $\mathbf{x}$ with d features, a weight vector $\mathbf{w}$, and a bias b. The perceptron takes all the inputs, multiplies each one by its weight, adds them up, and adds the bias:

$$a = \mathbf{w}^T\mathbf{x} + b = \sum_{i=1}^{d} w_i\, x_i + b$$

Then it checks the sign of a to decide the class:

$$\hat{y} = \operatorname{sign}(a) = \begin{cases} +1 & \text{if } a \geq 0, \\ -1 & \text{if } a < 0. \end{cases}$$

So $\mathbf{w}$ and b make a line (or hyperplane in higher dimensions) that splits the space into two sides. Points on one side get $+1$, points on the other side get -1. We call the real answer $y \in \{-1, +1\}$.

Step 2. We need to know if the perceptron gave the right answer or not. We can do this by multiplying y with the activation:

$$y\,(\mathbf{w}^T\mathbf{x} + b) > 0 \implies \text{correct}, \qquad y\,(\mathbf{w}^T\mathbf{x} + b) \leq 0 \implies \text{wrong}.$$

Reason: if y and a have the same sign (both positive or both negative), then the perceptron agreed with the true label, and the product will be positive. But if they have different signs, the product will be negative, which means there is an error.

Step 3. When the product $y\,(\mathbf{w}^T\mathbf{x} + b)$ is zero or negative, it means we have a wrong answer. In that case we need to change $\mathbf{w}$ and b to try to fix it. We want the product to become positive after the update.

If the answer is already correct, we do not touch anything.

Step 4. Here is the update rule we use. We choose a small number $\eta > 0$ (the learning rate). When the perceptron is wrong ($y\,(\mathbf{w}^T\mathbf{x} + b) \leq 0$), we update like this:

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} + \eta\, y\, \mathbf{x}, \qquad b_{\text{new}} = b_{\text{old}} + \eta\, y.$$

If it is correct, we do nothing.

Adding $\eta\, y\, \mathbf{x}$ to $\mathbf{w}$ pushes the activation a toward the correct sign, and the bias update does the same independently of the input.

Step 5. Now we need to prove that this update actually makes things better. We want to show that $y\,(\mathbf{w}^T\mathbf{x} + b)$ gets bigger after we apply the rule.

Let $M_{\text{old}} = y\,(\mathbf{w}_{\text{old}}^T\mathbf{x} + b_{\text{old}})$. We put the new $\mathbf{w}$ and b in:

$$\begin{aligned} M_{\text{new}} &= y\big((\mathbf{w}_{\text{old}} + \eta\, y\, \mathbf{x})^T\mathbf{x} + (b_{\text{old}} + \eta\, y)\big) \\ &= y\,(\mathbf{w}_{\text{old}}^T\mathbf{x} + b_{\text{old}}) + \eta\, y^2\, \|\mathbf{x}\|^2 + \eta\, y^2. \end{aligned}$$

We know that y is either $+1$ or -1, so $y^2 = 1$. This gives us:

$$M_{\text{new}} = M_{\text{old}} + \eta\,\big(\|\mathbf{x}\|^2 + 1\big).$$

Since $\eta > 0$ and $\|\mathbf{x}\|^2 + 1 \geq 1$, the extra term is always positive. So:

$$\boxed{M_{\text{new}} > M_{\text{old}}.}$$

This means every time we do an update, the margin gets bigger. One update might not be enough to fully correct the answer, but if the data can be separated by a line, the Perceptron Convergence Theorem says the algorithm will finish in a finite number of steps.

Step 6. So the final perceptron learning rule is:

$$\boxed{\mathbf{w} \leftarrow \mathbf{w} + \eta\, y\, \mathbf{x}, \qquad b \leftarrow b + \eta\, y, \qquad \text{only when } y\,(\mathbf{w}^T\mathbf{x} + b) \leq 0.}$$

We go through the training data, and every time we find a sample that the perceptron classifies wrong, we apply this update. We keep going through the data again and again until there are no more mistakes.